Конфликт имен и std namespace

На этом уроке мы рассмотрим, что такое конфликт имен в языке C++ и как его решить с помощью пространств имен и оператора разрешения области видимости.

Оглавление:

1. Конфликт имен в C++
2. Пространство имен
3. Оператор разрешения области видимости ::

Конфликт имен в C++

Допустим, что вам нужно съездить к дальним родственникам в другой город. У вас есть только их адрес: *г. Ржев, ул. Вербовая, 13*. Попав в город Ржев, вы открываете Google Карты/Яндекс.Карты и видите, что есть две улицы с названием *Вербовая*, еще и в противоположных концах города! Какая из них нужна вам? Если у вас нет никакой дополнительной информации (например, вы знаете, что их дом находится возле аптеки или школы), вам придется позвонить им и спросить. Чтобы подобной путаницы не возникало, все названия улиц в городе должны быть уникальными.

Аналогично и в языке C++ все идентификаторы (имена переменных/функций/классов и т.д.) должны быть уникальными. Если в вашей программе находятся два одинаковых идентификатора, то будьте уверены, что ваша программа не скомпилируется: вы получите ошибку **конфликта имен**.

Пример конфликта имен:

a.cpp:

```
#include <iostream>

void doSomething(int x)
{
    std::cout << x;
}
```

b.cpp:

```
#include <iostream>

void doSomething(int x)
{
    std::cout << x * 2;
}
```

main.cpp:

```
void doSomething(int x); // предварительное объявление функции doSomething()

int main()
{
    doSomething(5);

    return 0;
}
```

По отдельности файлы a.cpp, b.cpp и main.cpp скомпилируются. Однако, если a.cpp и b.cpp разместить в одном проекте — произойдет конфликт имен, так как определение функции doSomething() находится сразу в обоих файлах.

Большинство конфликтов имен происходят в двух случаях:

- Файлы, добавленные в один проект, имеют функцию (или глобальную переменную) с одинаковыми именами (ошибка на этапе **линкинга**).
- Файл .cpp подключает **заголовочный файл**, в котором идентификатор конфликтует с идентификатором из файла .cpp (ошибка на этапе компиляции).

Как только программы становятся больше, то и идентификаторов используется больше. Следовательно, вероятность возникновения конфликта имен значительно возрастает. Хорошая новость заключается в том, что язык C++ предоставляет достаточно механизмов для предотвращения возникновения конфликтов имен (например, **локальная область видимости** или пространства имен).

Пространство имен

В первых версиях языка C++ все идентификаторы из Стандартной библиотеки C++ (такие как **cin/cout** и т.д.) можно было использовать напрямую. Тем не менее, это означало, что любой идентификатор из Стандартной библиотеки C++ потенциально мог конфликтовать с именем, которое вы выбрали для ваших собственных идентификаторов. Код, который работал, мог внезапно получить конфликт имен при подключении нового заголовочного файла из Стандартной библиотеки C++. Или, что еще хуже, код, написанный по стандартам одной версии языка C++, мог уже не работать в новой версии языка C++. Чтобы устранить данную проблему, весь функционал Стандартной библиотеки C++ перенесли в специальную область — **пространство имен** (англ. ***«namespace»***).

Аналогично тому, как город гарантирует, что все улицы в его пределах имеют уникальные названия, так и пространство имен гарантирует, что все его идентификаторы — уникальны.

Таким образом, `std::cout` состоит из двух частей: идентификатор `cout` и пространство имен `std`. Весь функционал Стандартной библиотеки C++ определен внутри пространства имен `std` (сокр. от «***st**andar**d***»).

Мы еще поговорим о пространствах имен на следующих уроках, а также рассмотрим создание своего собственного пространства имен. Сейчас, главное, что вам нужно запомнить, — это то, что всякий раз, когда вы используете идентификаторы из Стандартной библиотеки C++ (например, cout), вы должны сообщать компилятору, что этот идентификатор находится внутри пространства имен std.

Правило: При использовании идентификаторов из пространства имен — указывайте используемое пространство имен.

Оператор разрешения области видимости ::

Самый простой способ сообщить компилятору, что определенный идентификатор находится в определенном пространстве имен — использовать **оператор разрешения области видимости** `::`. Например:

```
    std::cout << "Hello, world!";
```

Здесь мы сообщаем компилятору, что хотим использовать объект cout из пространства имен `std`.